JUPYTER NOTEBOOK

cleaning.ipynb

13 cells · python

INITIALIZATION & ENVIRONMENT SETUP

IN [11]:

```
# Import fundamental libraries for data manipulation

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

UNDERSTANDING THE DATA

IN [12]:

```
# Load the raw customer churn dataset into a pandas DataFrame
data = pd.read_csv('../data/bank_churn_data.csv')
df = pd.DataFrame(data)

df.head()
```

OUTPUT

	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfP
0	15634602	Hargrave	619	France	Female	42	2	0.00	1
1	15647311	Hill	608	Spain	Female	41	1	83807.86	1
2	15619304	Onio	502	France	Female	42	8	159660.80	3
3	15701354	Boni	699	France	Female	39	1	0.00	2
4	15737888	Mitchell	850	Spain	Female	43	2	125510.82	1

IN [13]:

```
# Output the matrix dimensions of the dataset (Rows, Columns)
print('Row, Col: ', df.shape)
print('='*175)

# Generate a high-level summary detailing data types, non-null counts
df.info()
print('='*175)

# Calculate descriptive statistics to identify structural ranges, standard deviations, and
potential outliers
df.describe()
```

OUTPUT

```
Row, Col:  (10000, 13)
===========================================================================================================
<class 'pandas.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 13 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   CustomerId       10000 non-null  int64
 1   Surname          10000 non-null  str
 2   CreditScore      10000 non-null  int64
 3   Geography        10000 non-null  str
 4   Gender           10000 non-null  str
 5   Age              10000 non-null  int64
 6   Tenure           10000 non-null  int64
 7   Balance          10000 non-null  float64
 8   NumOfProducts    10000 non-null  int64
 9   HasCrCard        10000 non-null  int64
 10  IsActiveMember   10000 non-null  int64
 11  EstimatedSalary  10000 non-null  float64
 12  Exited           10000 non-null  int64
dtypes: float64(2), int64(8), str(3)
memory usage: 1015.8 KB
===========================================================================================================
```

	CustomerId	CreditScore	Age	Tenure	Balance	NumOfProdu
count	1.000000e+04	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000
mean	1.569094e+07	650.528800	38.921800	5.012800	76485.889288	1.530200
std	7.193619e+04	96.653299	10.487806	2.892174	62397.405202	0.581654
min	1.556570e+07	350.000000	18.000000	0.000000	0.000000	1.000000
25%	1.562853e+07	584.000000	32.000000	3.000000	0.000000	1.000000
50%	1.569074e+07	652.000000	37.000000	5.000000	97198.540000	1.000000
75%	1.575323e+07	718.000000	44.000000	7.000000	127644.240000	2.000000
max	1.581569e+07	850.000000	92.000000	10.000000	250898.090000	4.000000

DATA INTEGRITY & QUALITY CLEANING

IN [14]:

```
# Convert CustomerId from numerical 'int64' to categorical string format
df['CustomerId'] = df['CustomerId'].astype(str)

# Verify successful conversion of the target feature's data type
print('CustomerId Dtype: ', df['CustomerId'].dtype)
```

OUTPUT

```
CustomerId Dtype:  str
```

IN [15]:

```
# Check the dataset for duplicate IDs and full-row identical entries across all columns
print(f'Unique CustomerId: {len(df['CustomerId'].unique())}')
print('='*175)
print(df.duplicated().sum())
```

OUTPUT

```
Unique CustomerId: 10000
===========================================================================================================
0
```

IN [16]:

```
# Inspect categorical columns to flag trailing spaces, formatting issues, or typos
print("Unique Gender values:", df['Gender'].unique())
print('='*175)
print("Unique Geographic Regions:", df['Geography'].unique())
```

OUTPUT

```
Unique Gender values: <StringArray>
['Female', 'Male']
Length: 2, dtype: str
===========================================================================================================
Unique Geographic Regions: <StringArray>
['France', 'Spain', 'Germany']
Length: 3, dtype: str
```

IN [17]:

```
# Value Distribution Validation
for col in ['HasCrCard', 'IsActiveMember', 'Exited']:
    print(f"\nValue distribution for column: {col}")
    print(df[col].value_counts())
```

OUTPUT

```
Value distribution for column: HasCrCard
HasCrCard
1    7055
0    2945
Name: count, dtype: int64

Value distribution for column: IsActiveMember
IsActiveMember
1    5151
0    4849
Name: count, dtype: int64

Value distribution for column: Exited
Exited
0    7963
1    2037
Name: count, dtype: int64
```

FEATURE ENGINEERING

IN [18]:

```
# Discretize 'Age' into categorical age brackets to simplify demographic analysis
df['AgeGroup'] = pd.cut(df['Age'],
                        bins=[18, 30, 45, 60, 100],
                        labels=['Young Adults', 'Adults', 'Older Adults', 'Seniors'],
                        include_lowest=True
                        )

# Segment 'CreditScore' based on standard consumer credit risk tiers
df['CreditScoreTier'] = pd.cut(df['CreditScore'],
                               bins=[300, 579, 669, 739, 799, 850],
                               labels=['Poor', 'Fair', 'Good', 'Very Good',
'Exceptional'],
                               include_lowest=True
                               )

# Segment 'Balance' to isolate zero-balance customers from low, mid, and high-capital
holders
df['BalanceSegment'] = pd.cut(df['Balance'],
                              bins=[-1, 0, 50000, 150000, 300000],
                              labels=['Zero', 'Low', 'Medium', 'High'],
                              include_lowest=True
                              )

# Divide 'EstimatedSalary' into 4 equal quarters (Quantiles)
df['SalaryGroup'] = pd.qcut(df['EstimatedSalary'],
                            q=4,
                            labels=['Low', 'Medium-Low', 'Medium-High', 'High']
                            )
```

IN [19]:

```
# Save the structured, cleaned, and feature-engineered DataFrame for downstream analysis
df.to_csv('../data/bank_churn_data_cleaned.csv', index=False)
```